Andy's LineMatcher Matching Stack

dark stack / deterministic fuzzy matching

This document describes exactly how the matcher works, what scoring functions are used, and how the final decision is made.

Overview

The matcher is deterministic. It does not use ML, embeddings, LLM reasoning, or a trained model.

The stack is:

1. normalize input strings
2. generate RC city variants
3. filter candidates by country
4. check manual alias overrides
5. score each candidate with three fuzzy heuristics
6. keep the best candidate per RC row
7. apply thresholds and confidence margin rules

Matching pipeline

For a single `shrep` row:

1. `Arrival Country` is normalized.
2. `Delivery City` is normalized.
3. The RC candidate pool is restricted to rows with the same normalized `Destination Country`.
4. If `city_aliases.csv` contains an approved override, that mapping wins immediately.
5. Otherwise each candidate gets a fuzzy score.
6. The best three candidates are retained for output and review.
7. The top score and its margin over the second candidate determine the final status.

Normalization layer

`normalize_basic`

Applied first to city names and country values:

- trims whitespace
- converts to uppercase
- removes diacritics with Unicode NFKD normalization
- strips non-alphanumeric characters to spaces
- collapses repeated whitespace

Example:

- `Dubai, UAE` -> `DUBAI UAE`
- `Petah-Tikva` -> `PETAH TIKVA`

`normalize_city`

Builds on `normalize_basic` and then:

- removes generic location words such as `CITY`, `TOWN`, `DISTRICT`, `PROVINCE`
- removes a trailing country token when present and when the city still has more than one token

Example:

- `Dubai, UAE` with country `AE` -> `DUBAI`
- `Makati City` -> `MAKATI`

`city_variants`

RC cells may contain multiple location fragments such as:

- `BERKELEY VALE; WARNERVALE NSW`

The matcher splits RC cities on:

- `;`
- `,`
- `/`
- `|`

Each part is normalized and stored as an additional searchable variant that still points to the same original RC row.

Candidate filtering

Before fuzzy scoring, the matcher narrows the pool to:

- candidates where normalized `Arrival Country == Destination Country`

This is a strong safety filter and prevents many wrong matches.

If there are no same-country candidates, the code can optionally fall back to cross-country search, but that behavior is disabled by default.

Alias override

`city_aliases.csv` is a manual memory layer.

If an approved pair exists for:

- normalized source country
- normalized source city

then the matcher returns:

- `status = auto_matched`
- `method = alias`
- `score = 100`

without running fuzzy ranking.

Fuzzy scoring functions

The matcher computes three scores and takes the maximum.

1. Gestalt similarity

Function:

- `sequence_score(left, right)`

Implementation:

- Python `difflib.SequenceMatcher`

Formula:

- `score_seq = 100 * SequenceMatcher(None, left, right).ratio()`

This is Ratcliff-Obershelp style similarity. It rewards large shared character blocks and works well for spelling variants such as:

- `SHARJACH` vs `SHARJAH`
- `PETAH TIKVA` vs `PETAKH TIKVA`

2. Token sort similarity

Function:

- `token_sort_score(left, right)`

Formula:

- tokenize both strings on spaces
- sort tokens alphabetically
- join back into strings
- apply `sequence_score`

So:

- `PETAH TIKVA`
- `TIKVA PETAH`

become the same sorted token order before the Gestalt score is computed.

This helps when both strings contain the same words but in different order.

3. Subset / containment heuristic

Function:

- `subset_score(left, right)`

Logic:

- convert both strings to token sets
- identify the shorter and longer set
- if the shorter set is a subset of the longer set, assign a high score

Formula:

- if `shorter ⊆ longer`

- `token_gap = len(longer) - len(shorter)`
- `score_subset = max(90, 97 - 2 * token_gap)`
- otherwise `score_subset = 0`

This captures cases where one city is essentially contained inside a longer RC form:

- `WARNERVALE`
- `BERKELEY VALE WARNERVALE NSW`

Final fuzzy score

Function:

- `city_score(left, right)`

Formula:

```
score_final = max(
  score_seq,
  score_token_sort,
  score_subset
)
```

Ranking and de-duplication

Multiple RC variants can point to the same RC row.

To avoid the same RC row appearing several times in the ranking:

- candidates are grouped by `rc_row_id`
- only the highest score for that RC row is kept

Then rows are sorted descending by score.

Decision rules

After ranking:

- `top_score` = score of best candidate
- `second_score` = score of second candidate, or `0`
- `margin = top_score - second_score`

Decision logic:

```
if top_score >= auto_threshold and (top_score == 100 or margin >= min_margin):
    status = auto_matched
elif top_score >= review_threshold:
    status = review_needed
else:
    status = unmatched
```

Default thresholds:

- `auto_threshold = 90`
- `review_threshold = 75`
- `min_margin = 3`

Why this works well for logistics city data

This stack is tuned for:

- spelling differences
- punctuation differences
- country suffixes
- multi-part RC city cells
- variant ordering
- safe human review when certainty is low

It is intentionally auditable. Every accepted match can be explained through:

- normalized source city
- candidate list
- chosen method
- score
- margin

What it is not

It is not:

- Levenshtein distance
- phonetic matching

- semantic matching
- embedding similarity
- machine learning

It is a deterministic fuzzy matcher with manual memory and thresholded review.